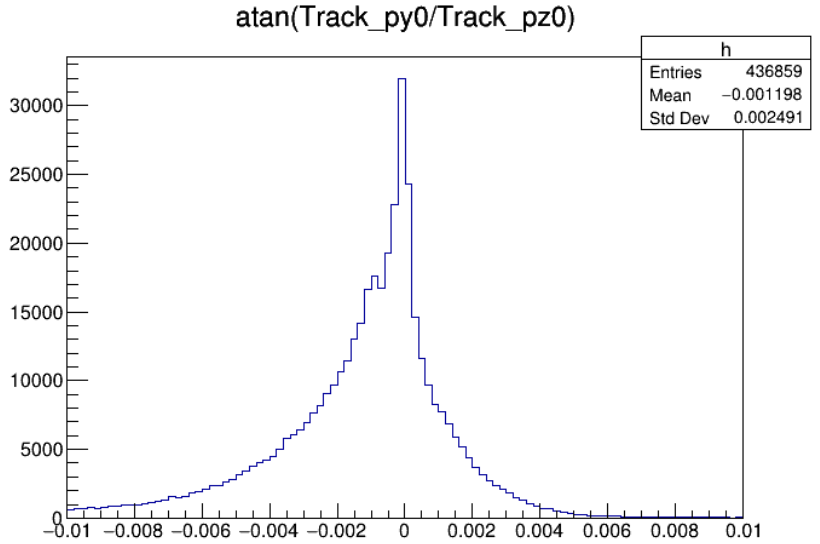
Progress on geometric rotation for FASER tracking

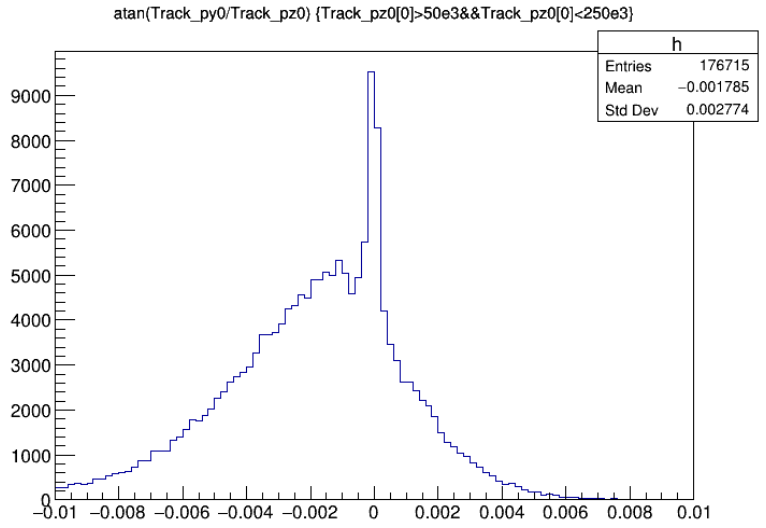
Saimeng Zhang，Xiaocong Ai
Zhengzhou University

Tracking Meeting, October 30, 2026

Motivation

- Track Parameterization near the Beam Axis
  - Forward tracking in FASER: Charged particles enter the tracking spectrometer predominantly along the beam-line direction (+z).
  - The standard angular parameterization becomes singular at ($\theta \to 0$), potentially affecting the numerical behavior of track reconstruction near the beam axis.
  - Unphysical peak for $\tan\theta_{y,st1} = 0$ is observed.(same as Mansoora and Tobias showed)
    - 100044 (μ+) : /eos/experiment/faser/data0/sim/mc24/particle_gun/100044/phy/s0013-r0022/FaserMC-MC24_PG_mupl_fasernu_5mrad_flukaE-100044-00000-00099-s0013-r0022-PHYS.root





Geometry Optimization Strategy

- Rotate geometry around global y by 90 deg (z->x)

```
RotationMatrix3 R = RotationMatrix3::Identity();
R.col(0) = Vector3(0, 0, -1);
R.col(1) = Vector3(0, 1, 0);
R.col(2) = Vector3(1, 0, 0);
Transform3 trafo(R);

Transform3 newTrafo = trafo*oldTrafo;
```

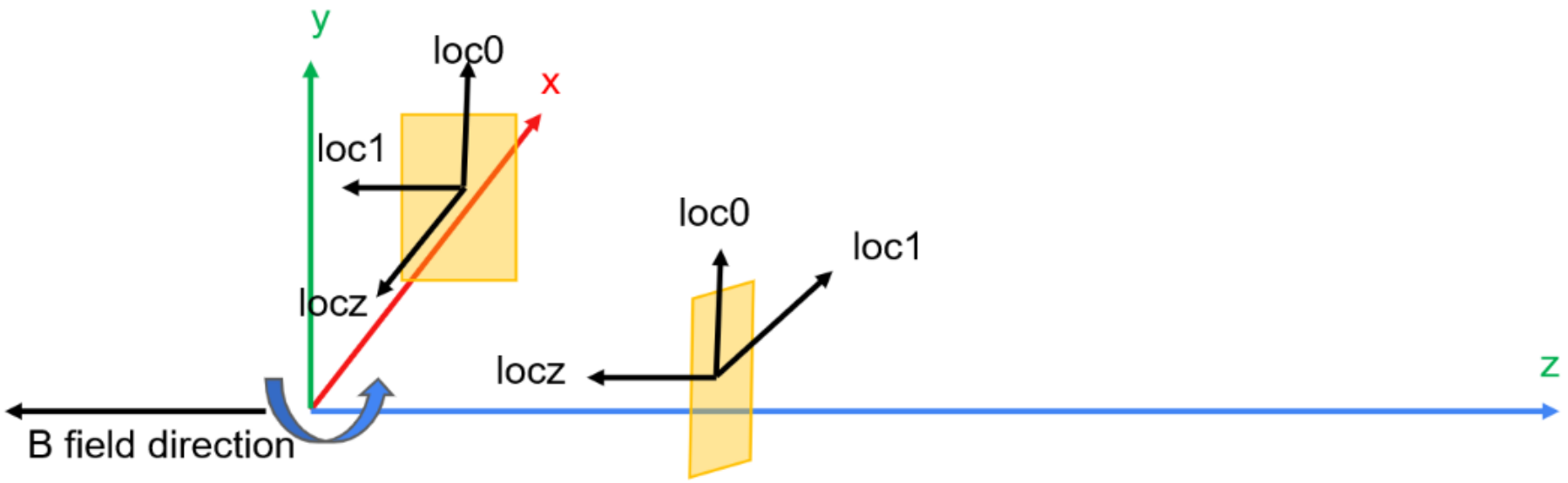
- B field direction

calypso/Tracking/Acts/FaserActsGeometry/FaserActsGeometry/FASERMagneticFieldWrapper.h

$$\vec{x'} = R \cdot \vec{x}$$

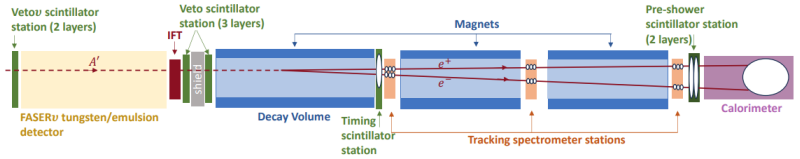
$$\vec{B'} = R \cdot \vec{B}$$

$$\frac{\mathrm{d}\vec{B'}}{\mathrm{d}\vec{x'}} = R \cdot \frac{\mathrm{d}\vec{B}}{\mathrm{d}\vec{x}} \cdot R^{-1}$$



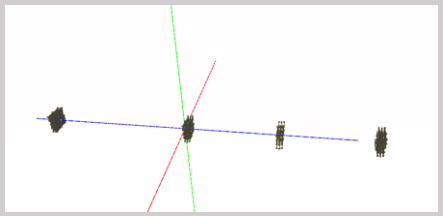
workflow Overview

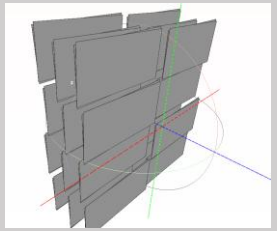
Original FASER geometry



Add transformation

ACTS geometry





Reconstruction chain
Seeding(initial Parameters) → CKF

calypso/Tracking/Acts/FaserActsGeometry/src/FaserActsTrackingGeometrySvc.cxx

- Retrieve each sub-detector
- trackingVolumeBuilder
  - makeVolumeBuilder() function

surfaceCfg & layerCfg & volumeCfg

⇩

build Layer→buildVolume→trackingVolume

⇩

trackingGeometryBuilder

- trackingGeometryBuilders

- Consistent coordinate convention:
  - InitialParameters provided by seeding process
  - CKF: target surface and refit process

FASER geometry → ACTS geometry

- **surfaceCfg & layerCfg & volumeCfg**: calypso/Tracking/Acts/FaserActsGeometry/src/FaserActsLayerBuilder.cxx
  - 4 stationsVolume:

    SiDetectorElement → FaserActsDetectorElement → surfaceCfg → ProtoLayer(gctx,surfaces) → layerCfg → volumeCfg

```
FaserActsDetectorElement::FaserActsDetectorElement(const TrackerDD::SiDetectorElement* detElem)
{
  m_detElement = detElem;
  const TrackerDD::SiDetectorDesign &design = detElem->design();
  double hlX = design.width()/2. * length_unit;
  double hlY = design.length()/2. * length_unit;

  auto rectangleBounds = std::make_shared<const Acts::RectangleBounds>(hlX, hlY);

  m_bounds = rectangleBounds;

  m_surface = Acts::Surface::makeShared<Acts::PlaneSurface>(rectangleBounds, *this);
```

Get layerCfg according to ProtoLayer pl

```
Acts::ProtoLayer pl(gctx, surfaces);
surfacecfg.position = trafo * Acts::Vector3(0, 0, layerZ);
if (std::abs(layerZInner) > std::abs(layerZOuter)) std::swap(layerZInner, layerZOuter);
transformNominal = trafo * Transform3(Translation3(0., 0., layerZ));
transformInner = trafo * Transform3(Translation3(0., 0., layerZInner));
transformOuter = trafo * Transform3(Translation3(0., 0., layerZOuter));
```

Add transformation here

Get volumeCfg according to layerCfg

```
  if (iPlane == 1) {
    volumeConfig.position = fgctx->geometryRotation ? Acts::Vector3(surfacecfg.position.x(), 0, 0) : Acts::Vector3(0, 0, surfacecfg.position.z(
  }
}
volumeConfig.length   = m_trackerDimensions;
volumeConfig.layerCfg = layerConfigs;
volumeConfig.name     = "Station_" + std::to_string(iStation);
```

  - Other volumes(buildScintVolume / buildDipoleVolume):

```
if(vetoManager!=nullptr){
for(unsigned int i=0; i<vetoManager->getNumTreeTops(); i++){
  auto vol0 = vetoManager->getTreeTop(i)->getLogVol();
  //get the shape params and translation
  const GeoBox* shape = dynamic_cast<const GeoBox*>(vol0->getShape());
  auto trans = vetoManager->getTreeTop(i)->getX();
  if(vetoManager->getTreeTop(i)->getNChildVols()==0){
    volumeConfigs.emplace_back(buildScintVolume(gctx, trans.translation().x(),trans.translation().y(),trans.translation().z(), shape->getXH
  }
  else{
    for(size_t j =0 ;j< vetoManager->getTreeTop(i)->getNChildVols();j++){
      auto childtrans=vetoManager->getTreeTop(i)->getXToChildVol(j);
      const GeoBox* childshape = dynamic_cast<const GeoBox*>(vetoManager->getTreeTop(i)->getChildVol(j)->getLogVol()->getShape());
      volumeConfigs.emplace_back(buildScintVolume(gctx, trans.translation().x() + childtrans.translation().x(),trans.translation().y() + ch
    }
  }
}
```

three Surface building(innerBoundary / centerBoundary / outerBoundary)

```
Transform3 transformInner(Translation3(xpos*1_mm, ypos*1_mm, (zpos-zhalflength+0.001)*1_mm));

std::shared_ptr<Acts::PlaneSurface> innerBoundary =
 Acts::Surface::makeShared<Acts::PlaneSurface>(
 transformInner, rBounds);

Transform3 transformCenter(Translation3(xpos*1_mm, ypos*1_mm, zpos*1_mm));
std::shared_ptr<Acts::PlaneSurface> centerBoundary =
 Acts::Surface::makeShared<Acts::PlaneSurface>(
 transformCenter, rBounds);

Transform3 transformOuter(Translation3(xpos*1_mm, ypos*1_mm, (zpos+zhalflength-0.001)*1_mm));

std::shared_ptr<Acts::PlaneSurface> outerBoundary =
 Acts::Surface::makeShared<Acts::PlaneSurface>(
 transformOuter, rBounds);
```

Add transformation here

```
if (fgctx->geometryRotation){
  surfacecfg.position = trafo * Acts::Vector3(0, 0, zpos);
  transformCenter = trafo * transformCenter;
  transformInner = trafo * transformInner;
  transformOuter = trafo * transformOuter;
```

```
aSurfaces.push_back(std::move(innerBoundary));
aSurfaces.push_back(std::move(centerBoundary));
aSurfaces.push_back(std::move(outerBoundary));
Acts::ProtoLayer pl(gctx, aSurfaces);
```

FASER geometry → ACTS geometry

- **ProtoLayer(gctx, surfaces)→layerCfg**

Extent: update the boundary ranges in all directions of this layer as each surface is traversed.

Key point: **transform (gctx)** associated with the surface

```
ProtoLayerT(const GeometryContext& gctx,
            const std::vector<std::shared_ptr<SurfaceType>>& surfaces,
            const Transform3& transformIn = Transform3::Identity()) {
  transform = transformIn;
  m_surfaces.reserve(surfaces.size());
  for (const auto& sf : surfaces) {
    m_surfaces.push_back(sf.get());
  }
  std::vector<const Surface*> constSurfaces;
  if constexpr (!IsConst) {
    constSurfaces.reserve(surfaces.size());
    for (auto* sf : m_surfaces) {
      constSurfaces.push_back(sf);
    }
    measureImpl(gctx, constSurfaces, extent, transform);
  } else {
    measureImpl(gctx, m_surfaces, extent, transform);
  }
}
```

```
void ProtoLayerBase::measureImpl(const GeometryContext& gctx,
                                 const std::vector<const Surface*>& surfaces,
                                 Extent& extent, const Transform3& transform) {
  for (const auto& sf : surfaces) {
    // To prevent problematic isInsidePolygon check for straw surfaces with only
    // one lseg
    int lseg = (sf->type() != Surface::Straw) ? 1 : 2;
    auto sfPolyhedron = sf->polyhedronRepresentation(gctx, lseg);
    const DetectorElementBase* element = sf->associatedDetectorElement();
    const auto* regSurface = dynamic_cast<const RegularSurface*>(sf);
    if (element != nullptr && regSurface != nullptr) {
      // Take the thickness in account if necessary
      double thickness = element->thickness();
      // We need a translation along and opposite half thickness
      Vector3 sfNormal = regSurface->normal(gctx, sf->center(gctx));
      for (const auto& dT : {-0.5 * thickness, 0.5 * thickness}) {
        Transform3 dtransform = transform * Translation3{dT * sfNormal};
        extent.extend(sfPolyhedron.extent(dtransform));
      }
      continue;
    }
    extent.extend(sfPolyhedron.extent(transform));
  }
}
```

point (four points) → plane

```
for (const auto& v2D : vertices2D) {
  vertices.push_back(transform(gctx) * Vector3(v2D.x(), v2D.y(), 0.));
}

// All of those can be described as convex
// @todo same as for Discs: coversFull is not the right criterium
// for triangulation
if (!isEllipse || !innerExists || !coversFull) {
  auto [faces, triangularMesh] =
      detail::FacesHelper::convexFaceMesh(vertices);
  return Polyhedron(vertices, faces, triangularMesh, exactPolyhedron);
```

Update the layer range based on the range of each point forming the plane

```
Extent Polyhedron::extent(const Transform3& transform) const {
  Extent extent;
  auto vtxs = vertices;
  std::transform(vtxs.begin(), vtxs.end(), vtxs.begin(), [&](auto& v) {
    auto vt = (transform * v);
    extent.extend(vt);
    return (vt);
  });
```

FASER geometry → ACTS geometry

- **transform(gctx) associated with the surface**

```
FaserActsDetectorElement::FaserActsDetectorElement(const TrackerDD::SiDetectorElement* detElem)
{
  m_detElement = detElem;
  const TrackerDD::SiDetectorDesign &design = detElem->design();
  double hlX = design.width()/2. * length_unit;
  double hlY = design.length()/2. * length_unit;

  auto rectangleBounds = std::make_shared<const Acts::RectangleBounds>(hlX, hlY);

  m_bounds = rectangleBounds;

  m_surface = Acts::Surface::makeShared<Acts::PlaneSurface>(rectangleBounds, *this);
```

The ACTS surface is associated with the corresponding FASER SiDetectorElement

```
const Acts::Transform3& Acts::Surface::transform(
    const GeometryContext& gctx) const {
  if (m_associatedDetElement != nullptr) {
    return m_associatedDetElement->transform(gctx);
  }
  return m_transform;
}
```

detectorElement → transform(gctx)

```
const Acts::Transform3&
FaserActsDetectorElement::transform(const Acts::GeometryContext& anygctx) const
{
  // any cast to known context type
  const FaserActsGeometryContext* gctx = anygctx.get<const FaserActsGeometryContext*>();

  // This is needed for initial geometry construction. At that point, we don't have a
  // consistent view of the geometry yet, and thus we can't populate an alignment store
  // at that time.
  if (gctx->construction) {
    // this should only happen at initialize (1 thread, but mutex anyway)
    return getDefaultTransformMutexed();
  }
  // unpack the alignment store from the context
  const FaserActsAlignmentStore* alignmentStore = gctx->alignmentStore;

  // no GAS, is this initialization?
  assert(alignmentStore != nullptr);

  // get the correct cached transform
  // units should be fine here since we converted at construction
  const Transform3* cachedTrf = alignmentStore->getTransform(this);

  assert(cachedTrf != nullptr);

  return *cachedTrf;
}
```

Get transform of the SiDetectorElement

```
const Acts::Transform3 *
FaserActsAlignmentStore::getTransform(const FaserActsDetectorElement *ade) const {
  return m_transforms.getTransform(ade);
}
void
FaserActsDetectorElement::storeTransform(FaserActsAlignmentStore* gas) const
{
//  Amg::Transform3D g2l
//        = m_detElement->getMaterialGeom()->getDefAbsoluteTransform()
//        * Amg::CLHEPTransformToEigen(m_detElement->recoToHitTransform());
  Amg::Transform3D g2l = m_detElement->transform() ;

  // need to make sure translation has correct units
  g2l.translation() *= length_unit;

  //if geometry rotation

  // any cast to known context type
  FaserActsGeometryContext gctx;
  if (gctx.geometryRotation) {
    Acts::RotationMatrix3 rot = Acts::RotationMatrix3::Identity();
    rot.col(0) = Acts::Vector3(0, 0, -1);
    rot.col(1) = Acts::Vector3(0, 1, 0);
    rot.col(2) = Acts::Vector3(1, 0, 0);
    Acts::Transform3 trafo(rot);
    g2l = trafo * g2l;
  }

  gas->setTransform(this, g2l);
```

Add transformation here

FASER geometry → ACTS geometry

- build Layer→buildVolume→trackingVolume

  calypso/Tracking/Acts/FaserActsGeometry/src/CuboidVolumeBuilder.cxx

  - buildLayer →buildVolume（according to info of surfaceCfg & layerCfg & volumeCfg）
  - Attention (If geometry rotation):
    - rotate the direction of binning (z → x) and the setting of binning boundaries

```
auto binningValue = fgctx->geometryRotation ? Acts::BinningValue::binX : Acts::BinningValue::binZ;
std::unique_ptr<const Acts::LayerArray> layArr(
    layArrCreator.layerArray(gctx, layVec, minMax.first, minMax.second,
                             Acts::BinningType::arbitrary, binningValue));
```

    - glueTrackingVolume

```
  negFace = Acts::BoundarySurfaceFace::negativeFaceXY;
  posFace = Acts::BoundarySurfaceFace::positiveFaceXY;

for (unsigned int i = 0; i < volumes.size() - 1; i++) {
  volumes[i + 1]->glueTrackingVolume(gctx, negFace, volumes[i].get(), posFace);
  volumes[i]->glueTrackingVolume(gctx, posFace, volumes[i + 1].get(), negFace);
}
```

Don't need to modify: it will automatically adjust according to changes in the local coordinate system.

Modification of reconstruction chain

- Some areas need to be consistent with the new geometric coordinates

  - Seeding
    - initialParams passing to CKF

```
Acts::BoundTrackParameters CircleFitTrackSeedTool::Seed::get_params(double origin, Acts::BoundSquareMatrix cov, bool backward)
```

    - covariance matrix of the initialParams

```
#initial_variance_inflation = [100, 100, 100, 100, 1000]
initial_variance_inflation = [100, 100, 100, 10000, 100000]
```

  - CKF
    - targetSurface

```
Acts::RotationMatrix3 rotation = Acts::RotationMatrix3::Identity();
rotation.col(0) = Acts::Vector3(0, 1, 0);
rotation.col(1) = Acts::Vector3(0, 0, -1);
rotation.col(2) = Acts::Vector3(-1, 0, 0);
Acts::Translation3 trans(targetZposition, 0., 0.);
Acts::Transform3 trafo(trans*rotation);
ATH_MSG_DEBUG("ACTS CKF target surface has center at " << trafo.transl
targetSurface = Acts::Surface::makeShared<Acts::PlaneSurface>(trafo);
```

    - Refit process (initialSurface & getParametersFromTrack)

```
Acts::RotationMatrix3 rotation = Acts::RotationMatrix3::Identity();
rotation.col(0) = Acts::Vector3(0, 1, 0);
rotation.col(1) = Acts::Vector3(0, 0, -1);
rotation.col(2) = Acts::Vector3(-1, 0, 0);
Acts::Translation3 trans(origin, 0., 0.);
Acts::Transform3 traf(trans*rotation);
std::shared_ptr<const Acts::Surface> pSurface = Acts::Surface::makeShared<Acts::PlaneSurface>(traf);

//@todo: the initialSurface should be targetSurface
GeneralFitterOptions options{
    gctx, mfContext, calibContext, &(*pSurface),
    Acts::PropagatorPlainOptions()};
```

```
Acts::BoundTrackParameters
KalmanFitterTool::getParametersFromTrack(const Trk::TrackParameters *inputParameters,
                                         const Acts::BoundVector& inputVector, double origin) const {
  using namespace Acts::UnitLiterals;
//  if geoRotation
  Acts::RotationMatrix3 rotation = Acts::RotationMatrix3::Identity();
  rotation.col(0) = Acts::Vector3(0, 1, 0);
  rotation.col(1) = Acts::Vector3(0, 0, -1);
  rotation.col(2) = Acts::Vector3(-1, 0, 0);
  Acts::Translation3 trans(origin, 0., 0.);
  Acts::Transform3 traf(trans*rotation);
  std::shared_ptr<const Acts::Surface> pSurface = Acts::Surface::makeShared<Acts::PlaneSurface>(traf);
```

Test the results of geometry rotation

- The codes after geometryRotation
- Scripts of running geometry and reconstruction in here:
  /afs/cern.ch/work/s/sazhang/public/FASER/xai_alma9/check_geometryRotation
- Test
  - Run geometry:

    calypso/Tracking/Acts/FaserActsGeometry/FaserActsGeometry/FaserActsGeometryContext.h
    - "geometryRotation" variable controls whether the geometry rotates

```
struct FaserActsGeometryContext {

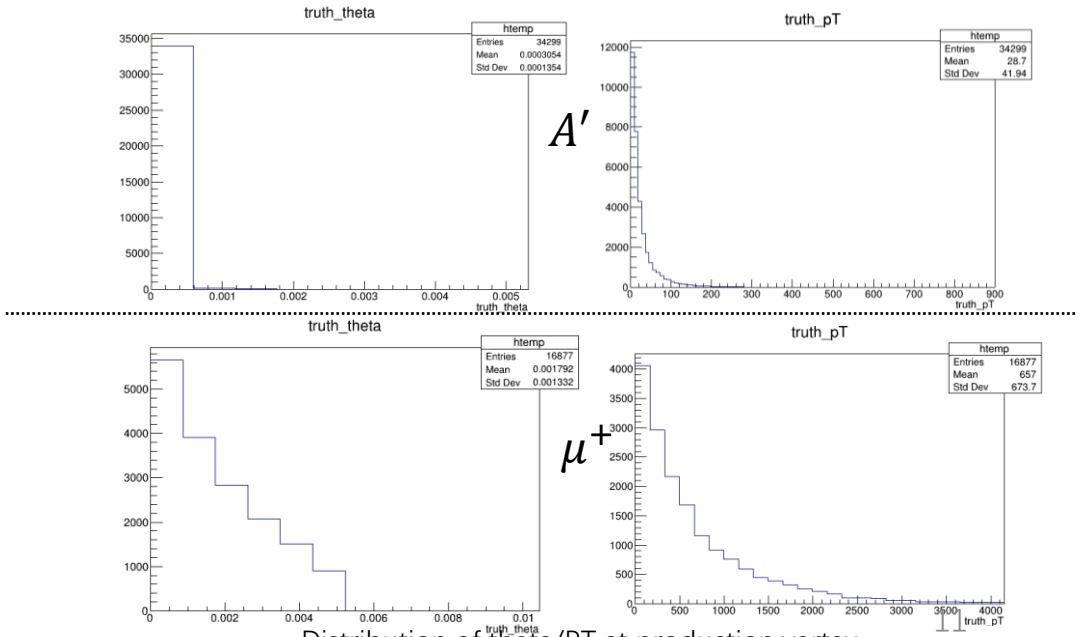
  bool construction{false};

  bool geometryRotation{true};
```

    - To do: switch needs to be added for the corresponding modifications in the reconstruction chain
  - Run reco (single event example: event21):
    - python faser_reco.py --isMC /eos/experiment/faser/data0/sim/mc24/foresee/110033/rdo/master_dev/FaserMC-MC24_FS_Aee_010MeV_1Em5-110033-00000-00001-master_dev-RDO.root –o "output file storage path" --skip 21 -n 1 -v > event21.log

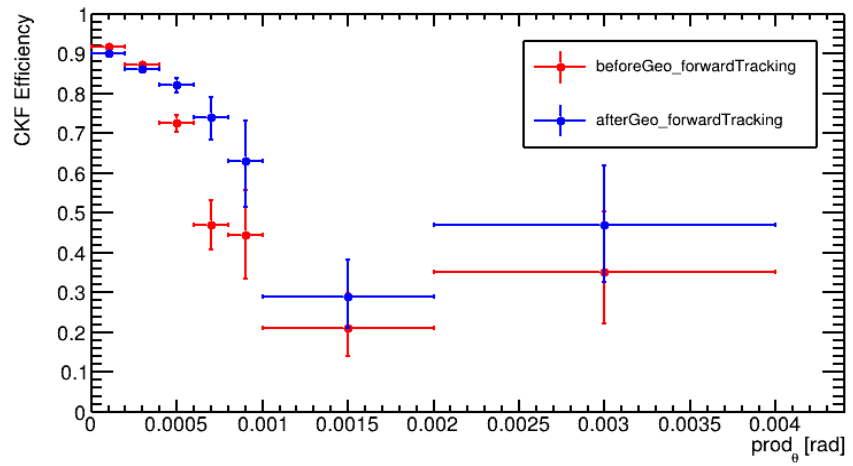
Comparison of beforeGeo & afterGeo

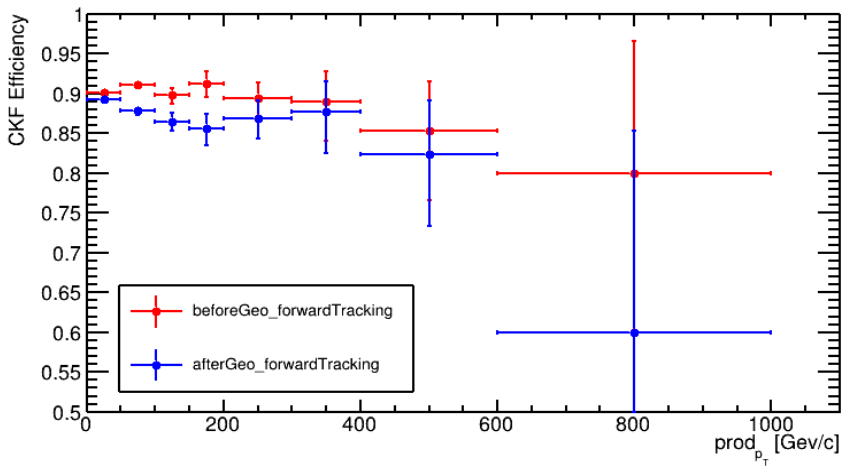
- **Sample:**
  - **DarkPhoton sample**: eos/experiment/faser/data0/sim/mc24/foresee/110033/rdo/master_dev/
  - **Muon sample**:/eos/experiment/faser/data0/sim/mc24/particle_gun/100044/phy/s0013-r0022/FaserMC-MC24_PG_mupl_fasernu_5mrad_flukaE-100044-00000-00099-s0013-r0022-PHYS.root
- **Tracking performance comparison**
  - CKF efficiency
    - Track-based Metrics (vs θ / P_T of charged particles at production vertex)
      - Efficiency = $\frac{nSeeds/nTracks^{\{reco,matched\}}}{nParticles^{\{fiducial,truth_nHits\geq 18\}}}$
    - Event-based Metrics (vs ΔR_1)
      - Efficiency = $\frac{nEvents^{\{nSeeds/nTracks\geq 2\}}}{nEvents^{\{nSeeds/nTracks\geq 1\}}}$
  - CKF momentum resolution
  - unphysical peak for tan θy = 0

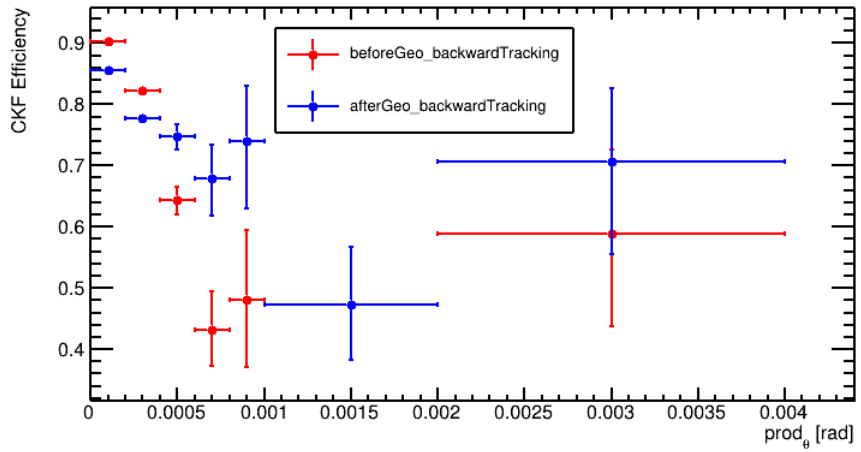


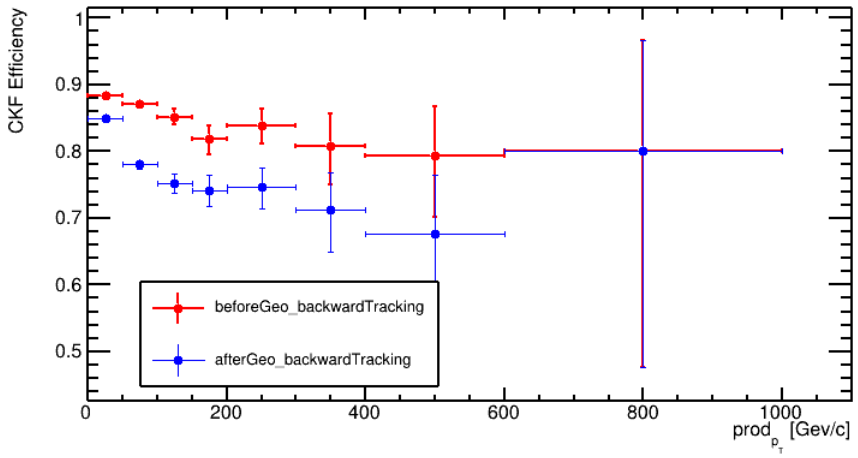
Distribution of theta/PT at production vertex

CKF Efficiency Comparison



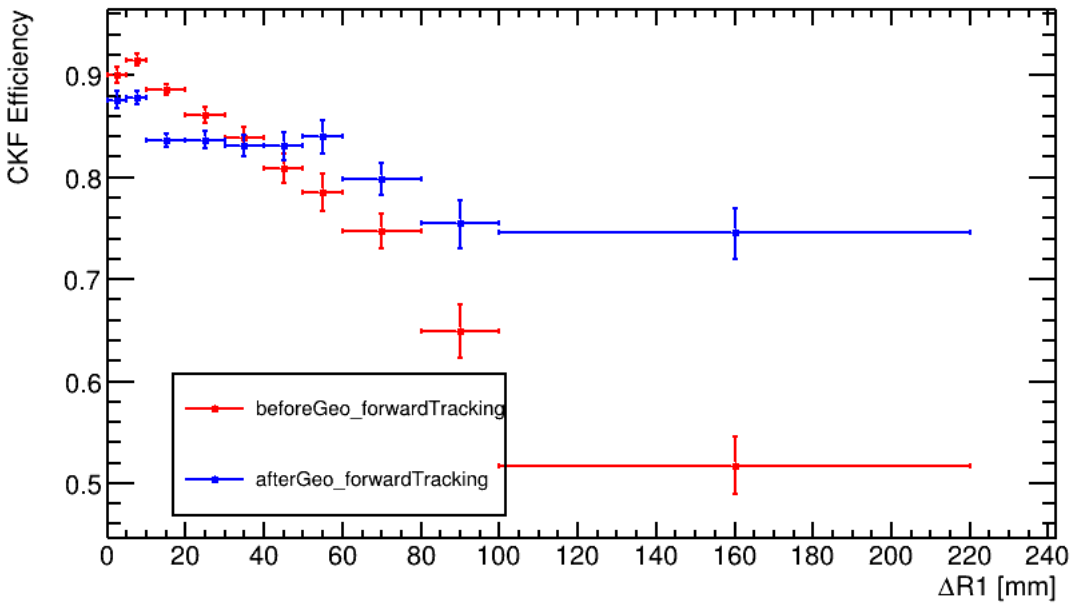


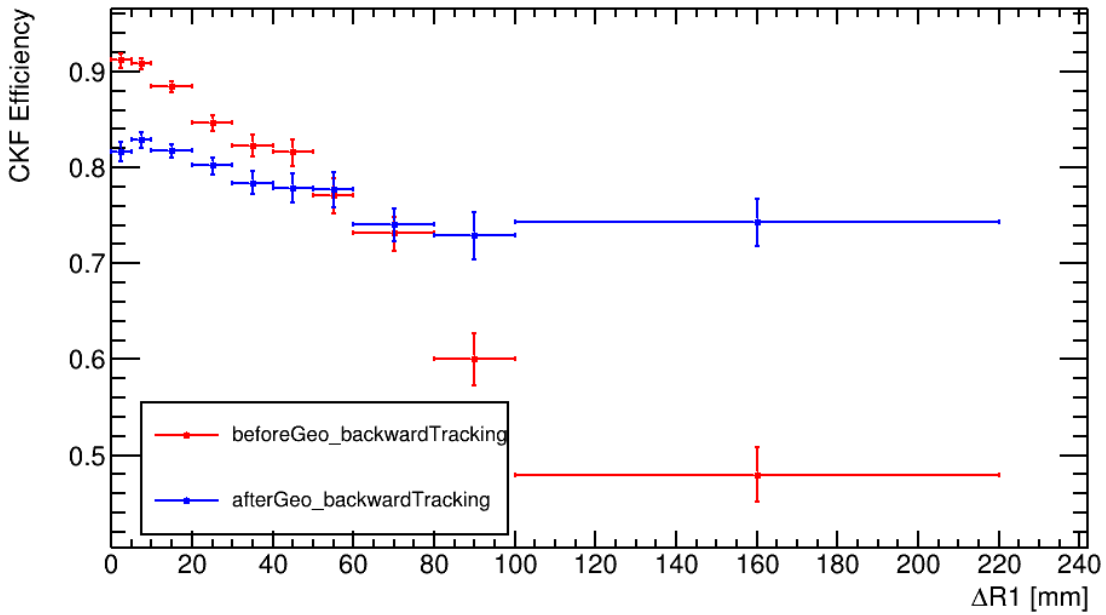




- Track-based efficiency Comparion(A' sample):
  - Whether forward tracking or backward tracking, the efficiency of afterGeo is slightly lower than beforeGeo, and this gap is more noticeable for backward tracking.
  - However, the overall efficiency difference is not particularly large (fluctuating at around 5%).

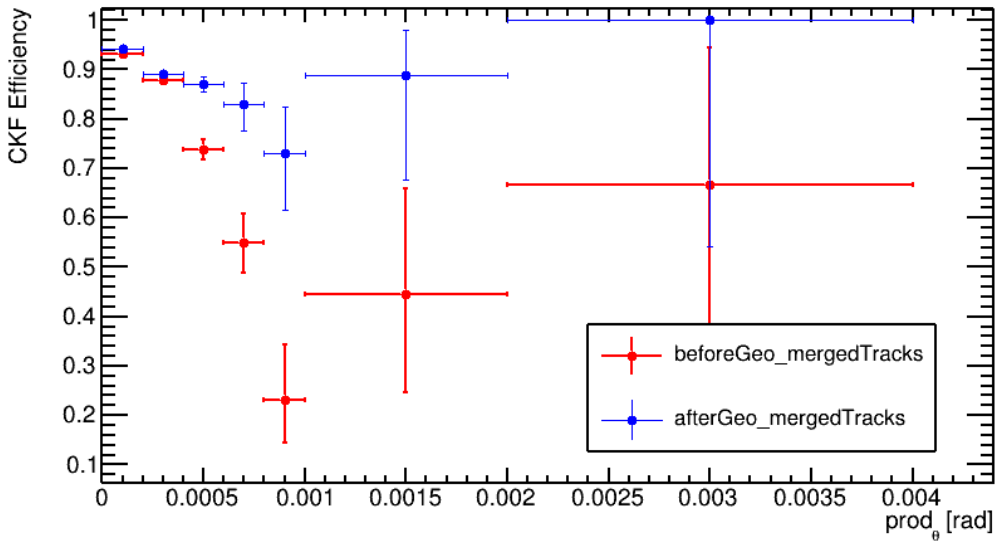
CKF Efficiency Comparison

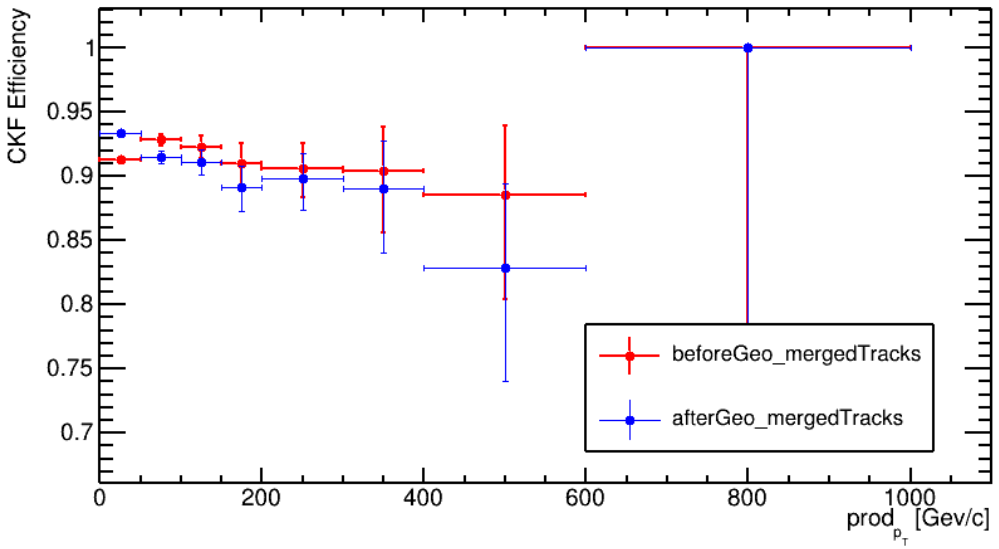


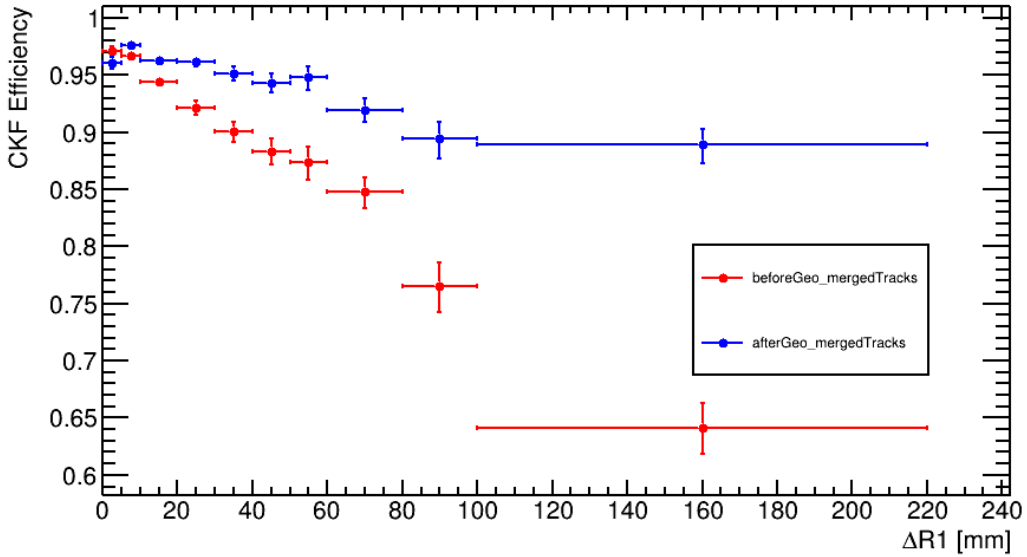


- Event-based efficiency vs ΔR_1 Comparion(A' sample):
  - Same as track-based efficiency, the afterGeo efficiency is slightly lower than beforeGeo in the low ΔR_1 range.
  - However, after geometry rotation, the tracking efficiency remains at a favorable level even in the large ΔR_1 range, showing an improvement compared with beforeGeo.

CKF Efficiency Comparison

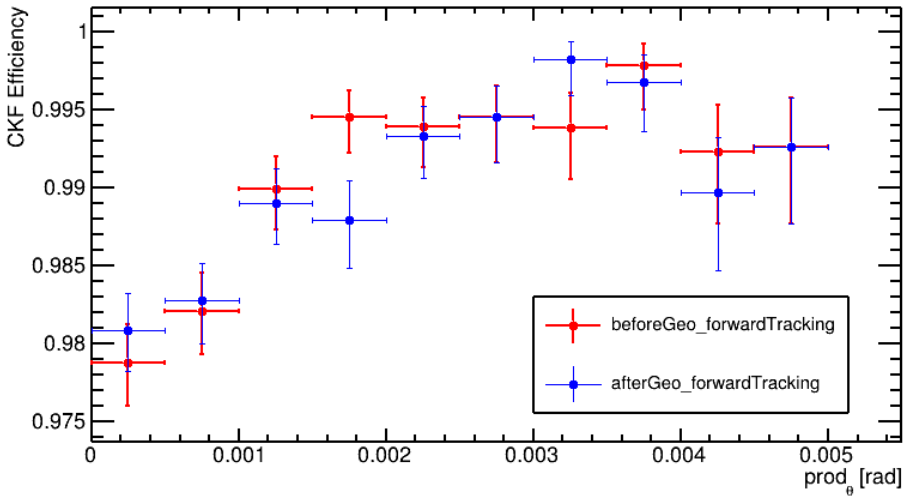


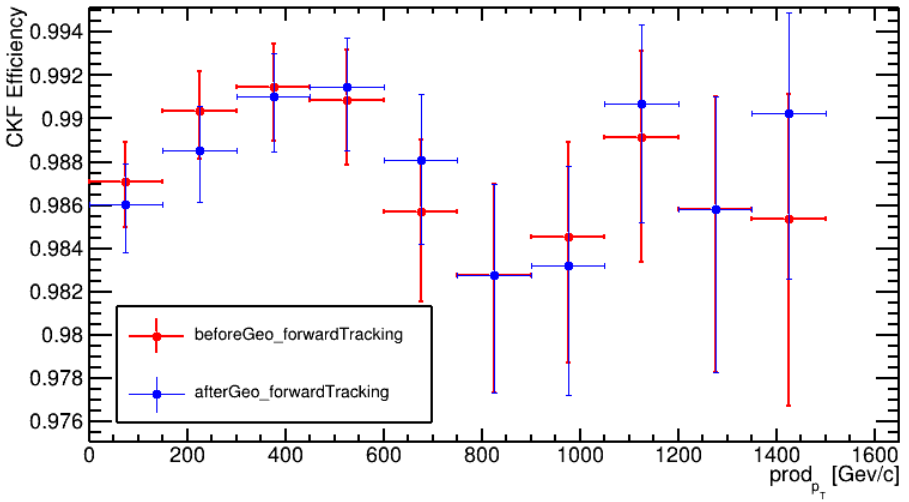


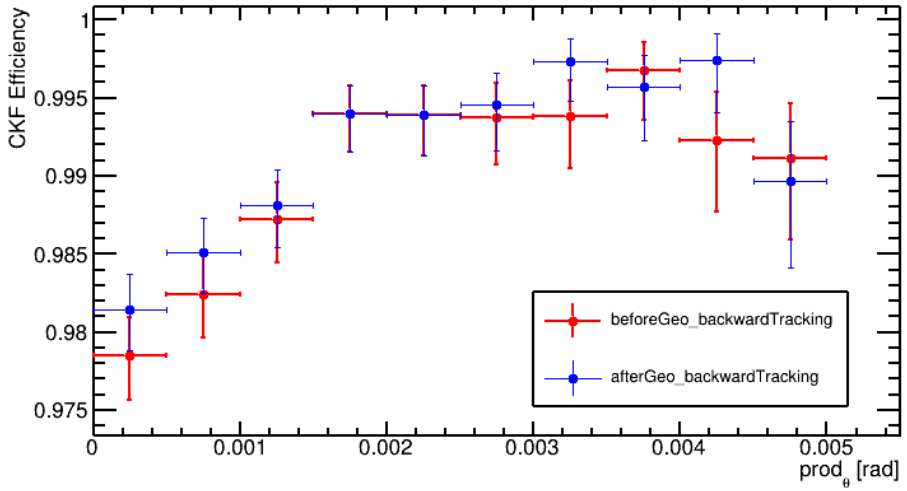


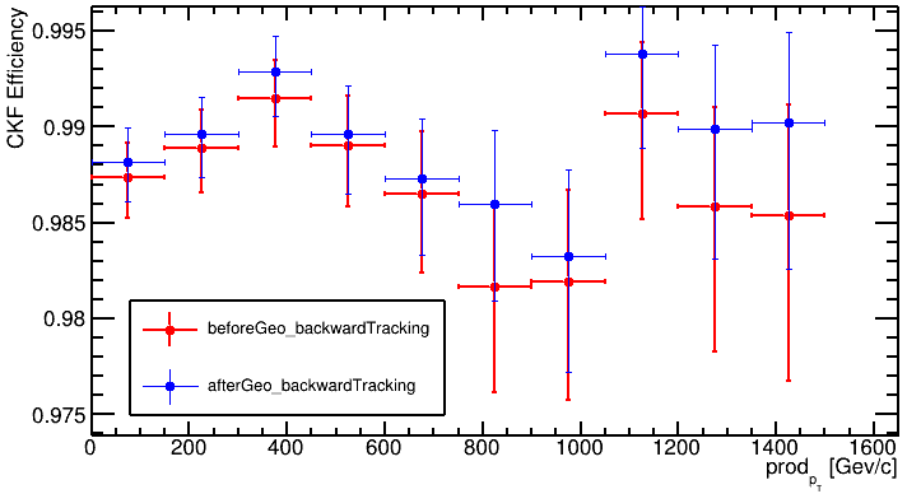
- Merged efficiency Comparion(A' sample):
  - The Merged efficiency of beforeGeo and afterGeo is basically comparable, and the efficiency of afterGeo is even slightly higher than beforeGeo.

CKF Efficiency Comparison







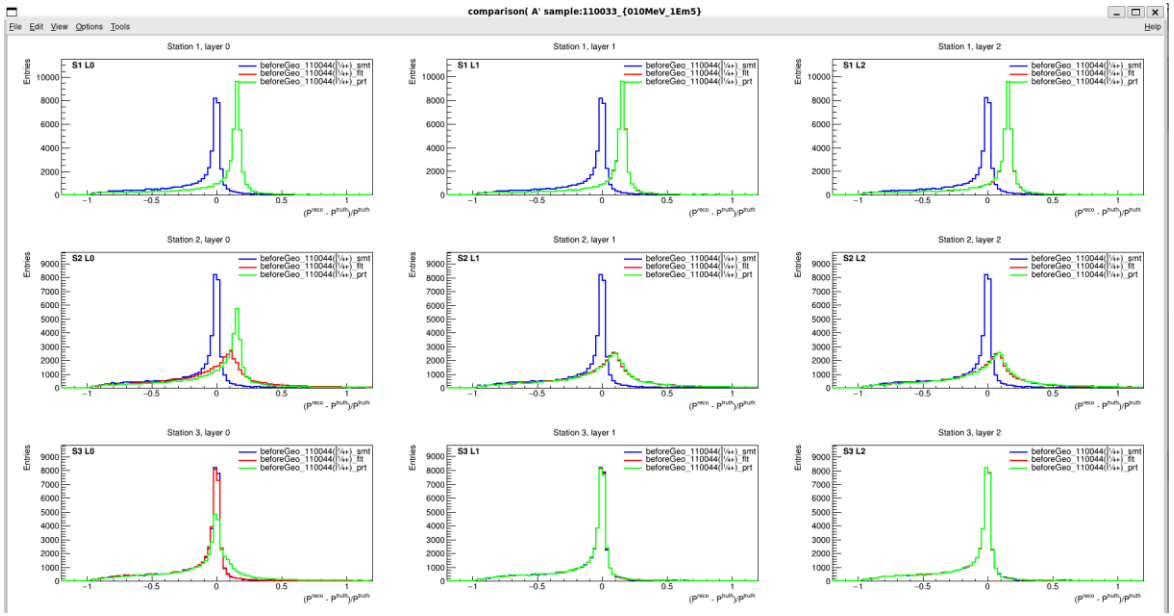


- Track-based efficiency Comparion(μ^+ sample):
  - For the single muon sample, the efficiency remains largely consistent before and after conversion, without significant fluctuations.

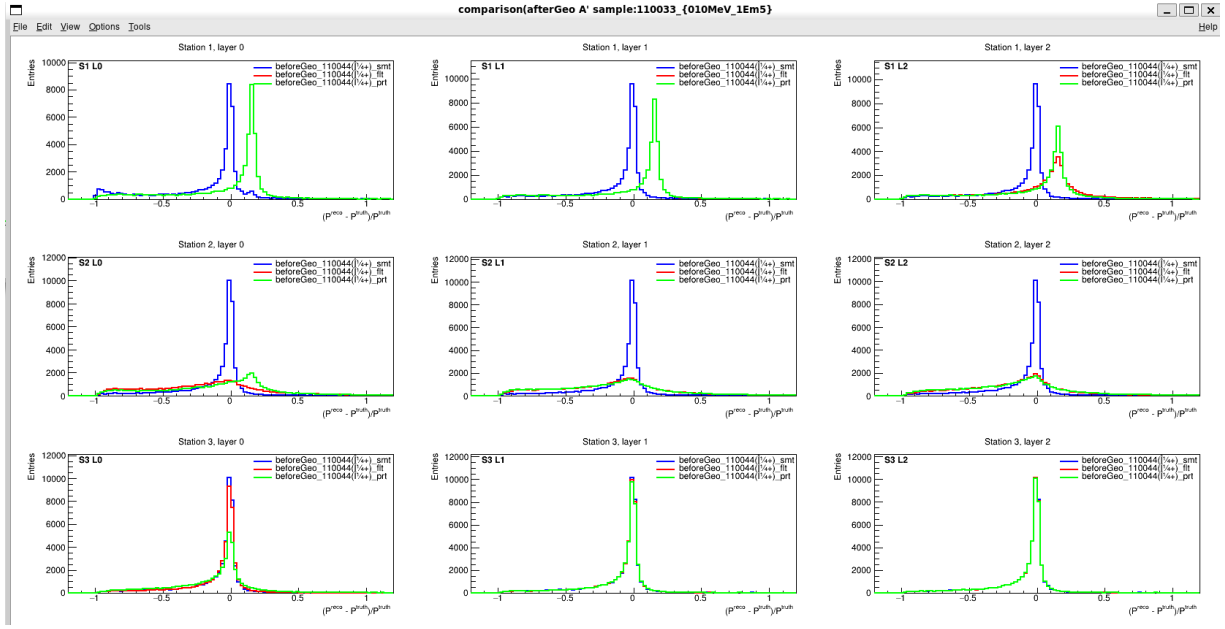
Momentum resolution Comparison

CKF perLayer P residual: $(P^{reco} - P^{truth}) / P^{truth}$ (green: predicted state; red:filtered state; blue: smoothed state)

beforeGeo



afterGeo

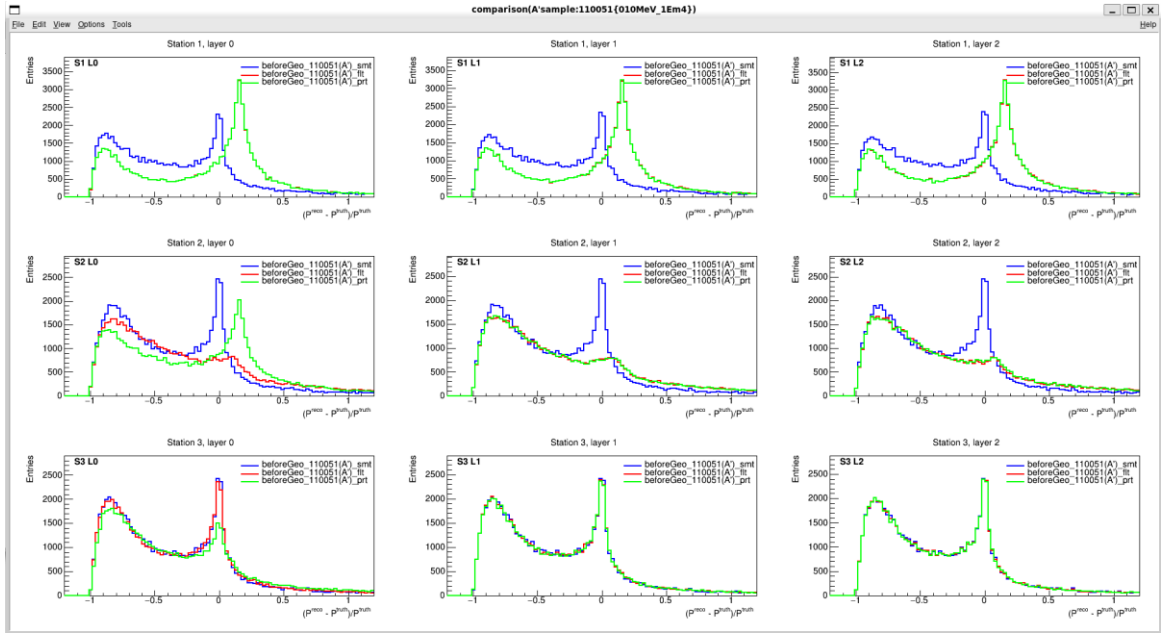


- A' sample {010MeV_1Em5-110033}:
  - The distribution tightens slightly after geometry rotation, yet the left-tailed distribution on the negative axis remains relatively broad both beforeGeo and afterGeo.

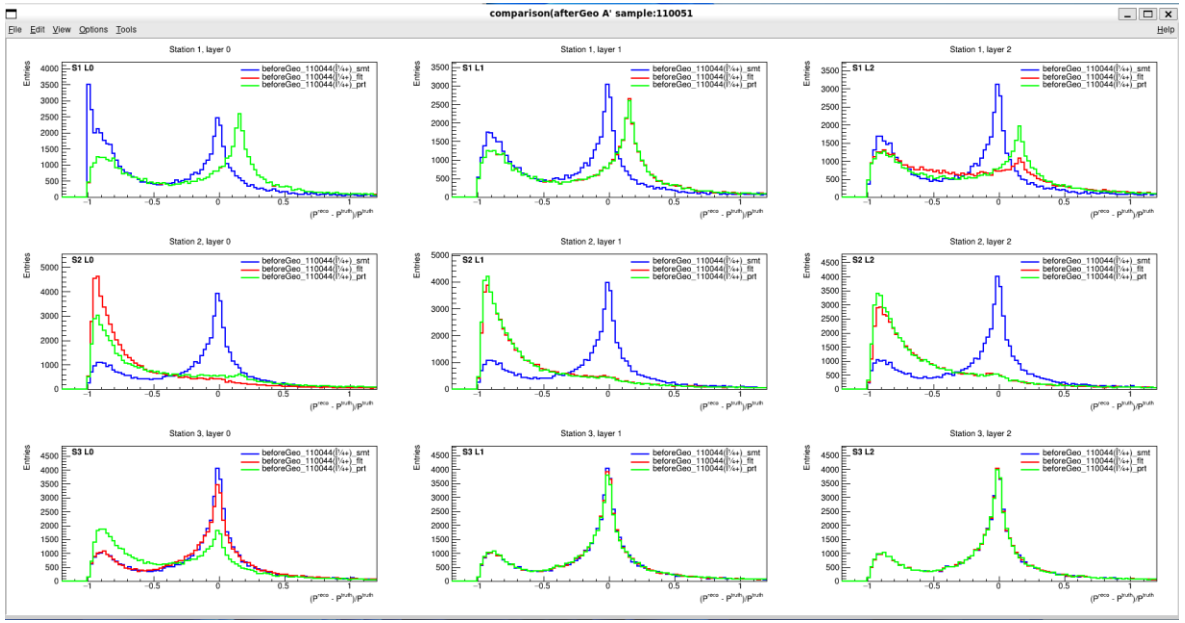
Momentum resolution Comparison

CKF perLayer P residual: $(P^{reco} - P^{truth}) / P^{truth}$ (green: predicted state; red:filtered state; blue: smoothed state)

beforeGeo



afterGeo

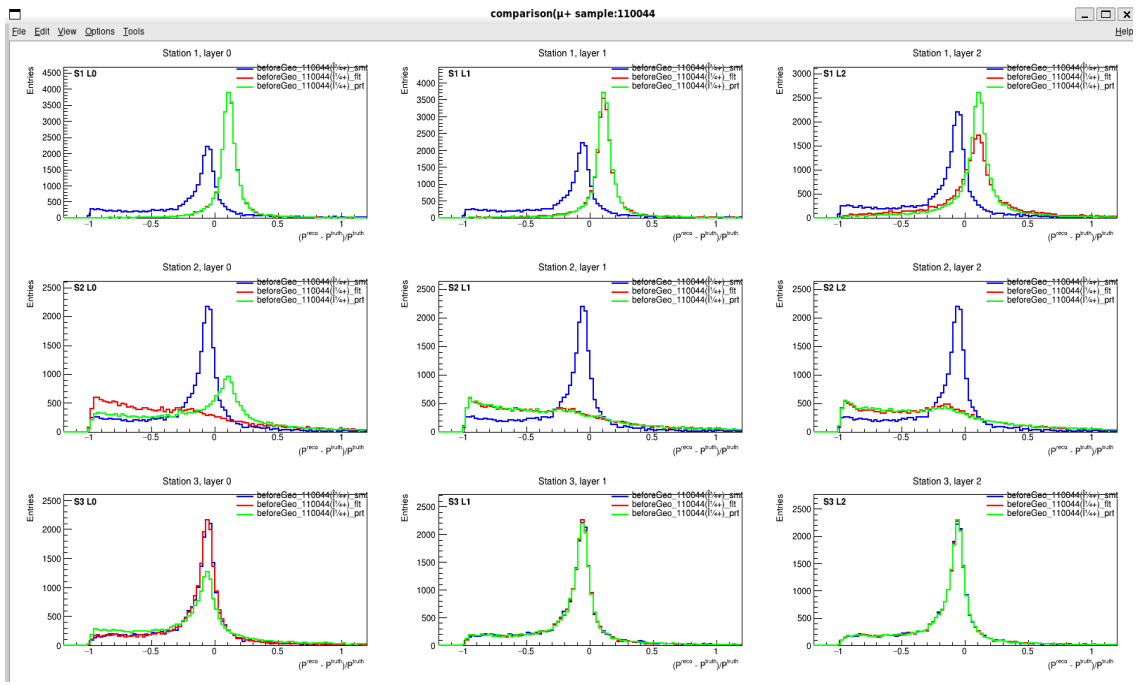


- A' sample {010MeV_1Em4-110051}:
  - Compared with 110033 sample, the anomalies of negative-axis tailing and abrupt peaks in 110051 sample are more obvious.
  - After geometric rotation, the abrupt peaks are slightly improved, but not completely eliminated.

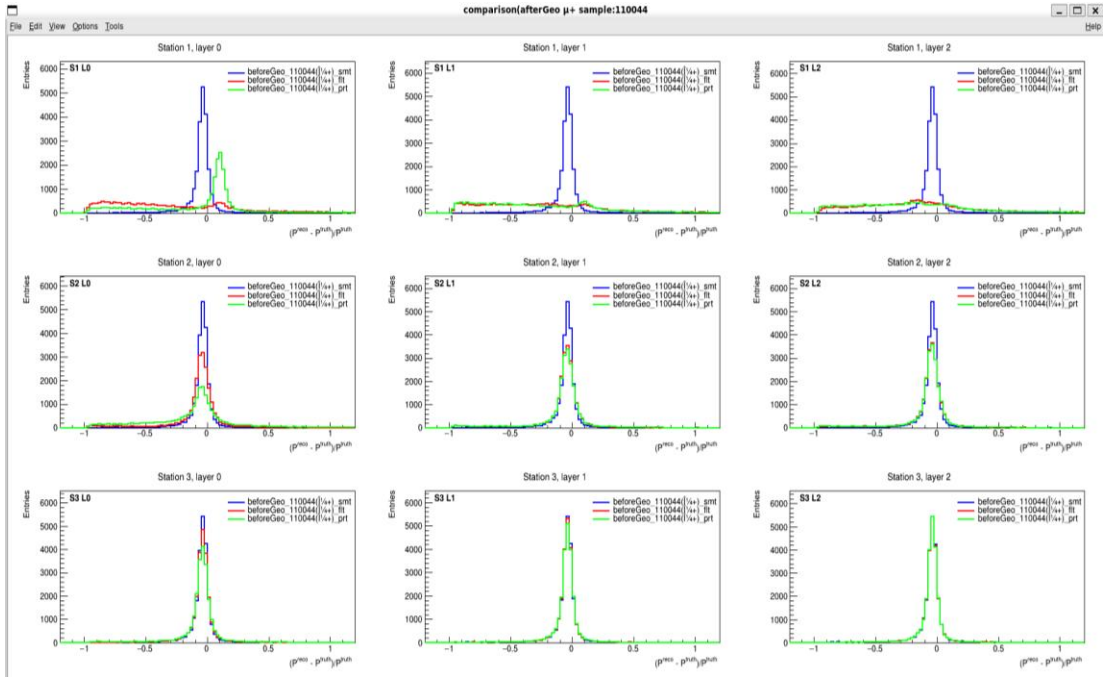
Momentum resolution Comparison

CKF perLayer P residual: $(P^{reco} - P^{truth}) / P^{truth}$ (green: predicted state; red:filtered state; blue: smoothed state)

beforeGeo



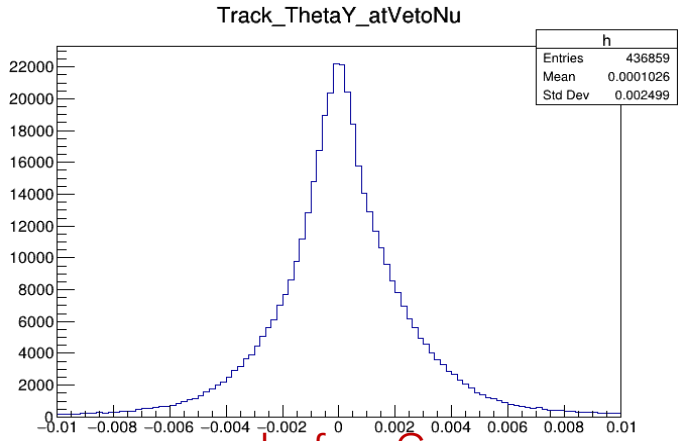
afterGeo



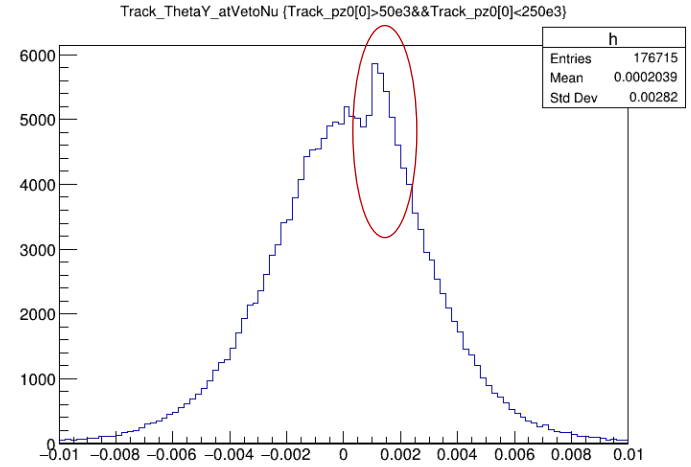
- μ^+ sample：
  - For the single muon sample, a wide tail on the negative side is also observed beforeGeo.
  - However, after geometric rotation, the tailing effect is mitigated and the overall distribution becomes more compact.

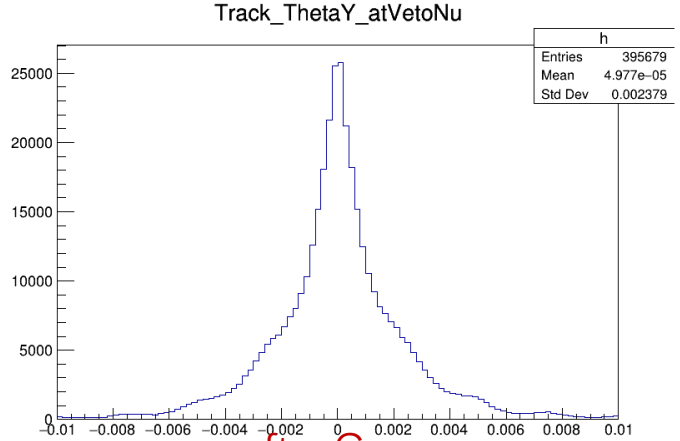
Unphysical peak for tan θy = 0

- Use 100044 (μ+) sample（500k events）: /eos/experiment/faser/data0/sim/mc24/particle_gun/100044/phy/s0013-r0022/FaserMC-MC24_PG_mupl_fasernu_5mrad_flukaE-100044-00000-00099-s0013-r0022-PHYS.root
- Compare the tan θy,VetoNu distribution results of beforeGeo (left) and afterGeo (right), and a certain range was compared.（Bottom: 50GeV<Preco<250GeV)

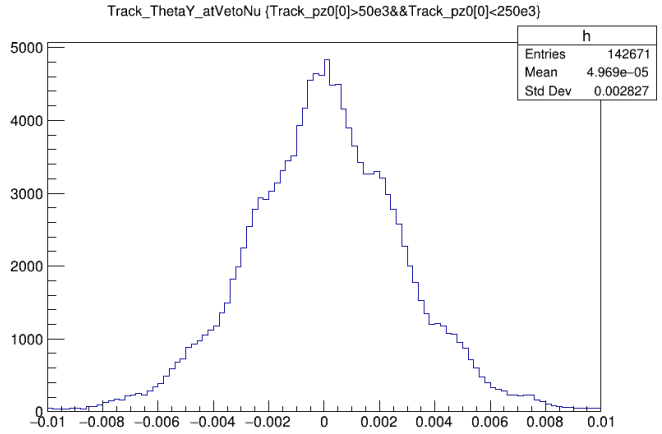


beforeGeo





afterGeo

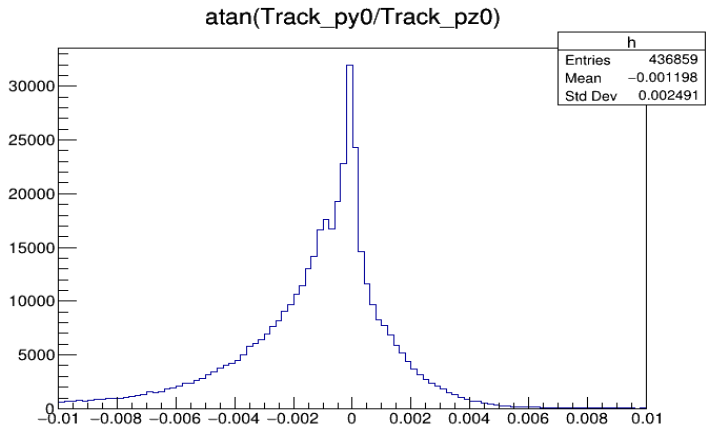


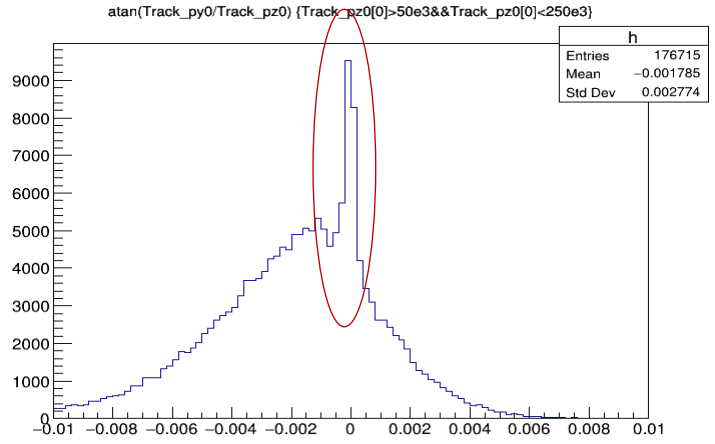
- Within a certain momentum range, an physical peak can be observed near tan θy,VetoNu = 0.
- After geometric rotation, the peak of tan θy,VetoNu near 0 is improved.

Unphysical peak for tan θy = 0

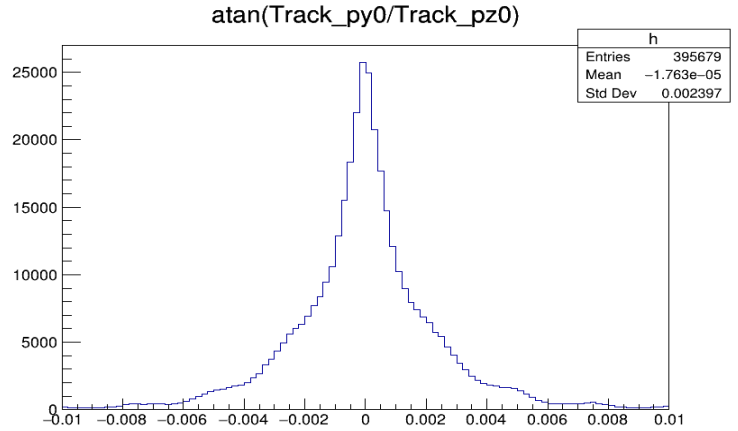
- Compare the tan θy, st1 distribution results of beforeGeo (left) and afterGeo (right), and a certain range was compared.（Bottom: 50GeV<Preco<250GeV）

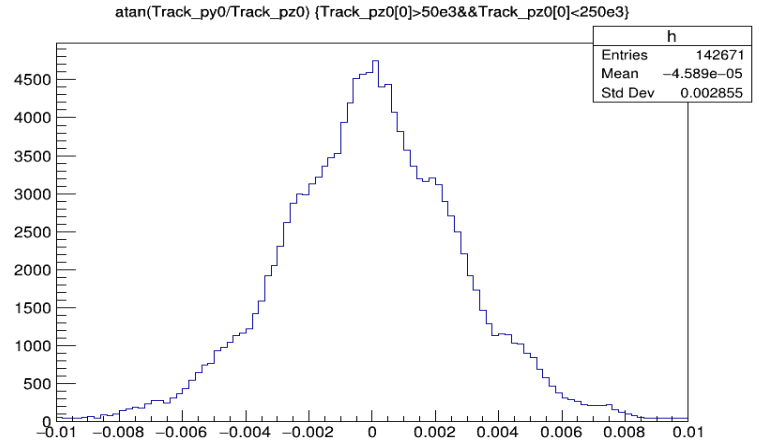
beforeGeo





afterGeo





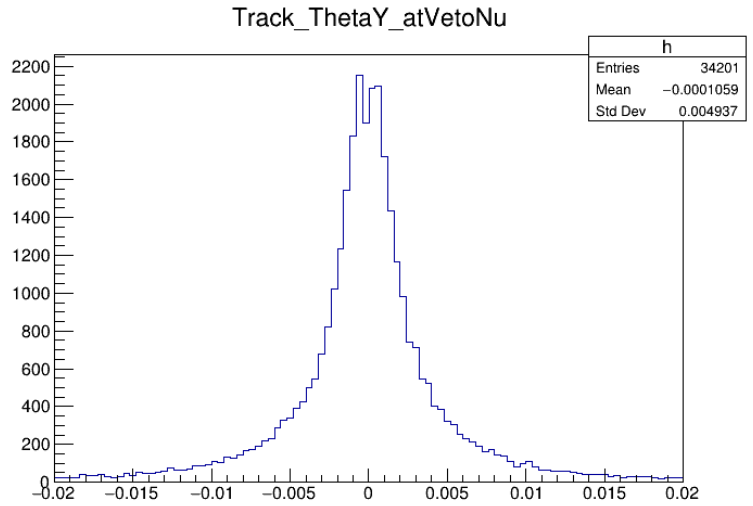
- Similarly, the sharp peak near tan θy, st1 = 0 is more prominent, which is improved after geometric rotation.

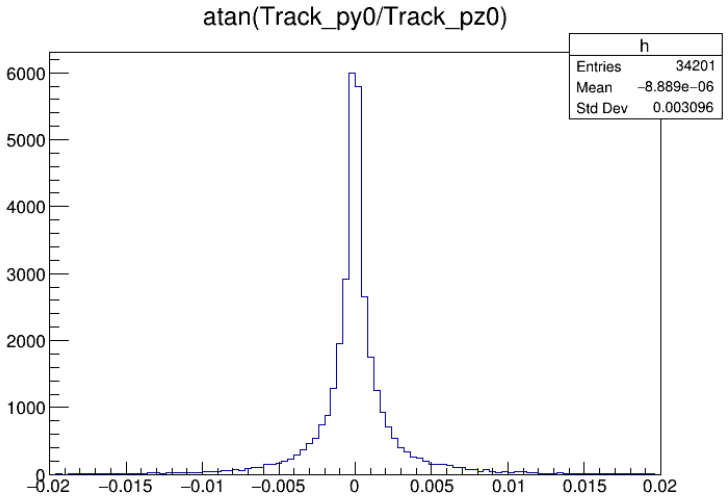
Summary

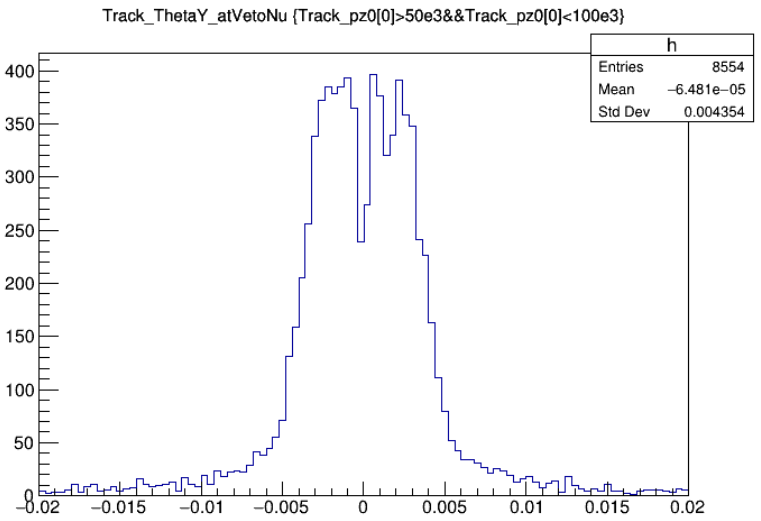
- **The overall performance of geometric rotation is good, especially for the single muon sample.**
  - The CKF track-based efficiency remains comparable to that of the original geometry, and the dependence of event-based efficiency on track separation becomes more stable after geometry rotation.
  - A clear improvement is observed for muon samples, while anomalous momentum residual structures persist in A' samples.
  - The unphysical peak at (tan thetaY=0) is significantly reduced.
- **Outlook**
  - Investigating the A' Momentum Resolution
    - Is it related to the seeding process ?

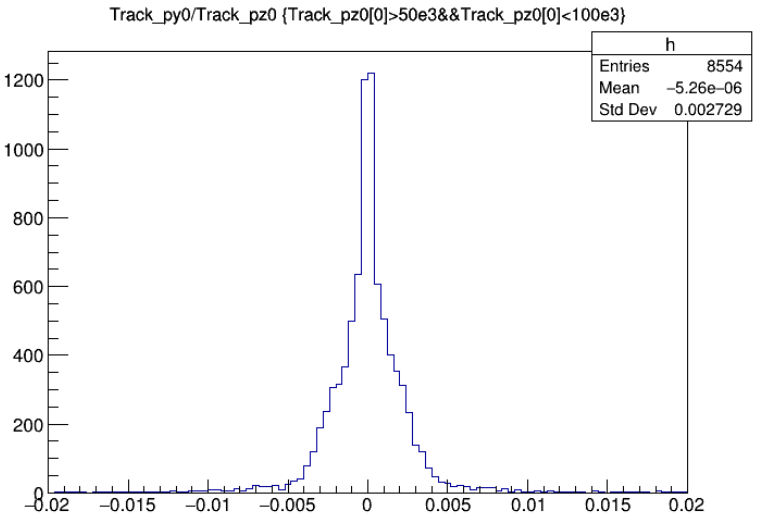
Backup

Distribution of tan θy for A'









Momentum resolution in seeding

